La mejora principal es separar la arquitectura técnica del prototipo de la arquitectura de colaboración con IA. Así el journal explica no solo cómo se procesaron los logs, sino también qué herramientas participaron, para qué se usó cada una, cómo se conectaron en el flujo de trabajo y qué decisiones permanecieron bajo control humano.

Conviene documentar dos niveles de evidencia: las herramientas y actividades descritas en el proceso de colaboración, y los componentes técnicos que aparecen en el código. No debe afirmarse que una herramienta ejecutó código, generó resultados o validó datos si eso no está documentado.

A continuación tienes una versión ampliada, con diagramas de arquitectura, responsabilidades por herramienta, flujo de trabajo, trazabilidad y controles de validación.

AI Collaboration Journal

Project: Employment Hero — Recruitment Management System (RMS) Telemetry Analysis
Candidate: Juan José Manriquez
Role: Intermediate Fullstack .NET Engineer
Consolidation date: October 19, 2026
Approach: Human-directed, AI-assisted engineering
Primary interaction interface: ChatGPT desktop application
Technical environment: Visual Studio Code, Python, local and web-based AI tools, and statistical analysis libraries

---

1. Purpose and scope

This journal documents how artificial intelligence was used to develop a telemetry-analysis prototype for the Recruitment Management System (RMS) engineering scenario.

Its purpose is to make the human–AI collaboration process traceable by distinguishing:

- Problems and constraints defined by the developer.
- Implementation proposals and explanations provided by AI.
- Tools used to support development, analysis, and communication.
- Technical and analytical decisions retained under human control.
- Limitations affecting the interpretation of results.

The prototype explores five telemetry signal categories:

- `TECH_DEBT`
- `SYSTEM_LAG`
- `OPERATIONAL_OVERHEAD`
- `LOW_ADOPTION`
- `FEATURE_PRESSURE`

The records in `system_signals.log` are **synthetic data generated to test the analytical workflow**. They are not verified production measurements from Employment Hero.

The work was conducted under the scenario's constraints: no additional budget, no additional headcount, and the capacity to prioritize no more than one or two initiatives.

The resulting prototype is intended to demonstrate engineering judgment, reproducible data processing, statistical exploration, and responsible AI-assisted development.

2. Executive summary of the collaboration

The developer defined the problem, selected the signal categories, established the resource constraints, and decided which quantitative analyses could help evaluate competing initiatives.

AI-assisted work supported the translation of those requirements into Python code, the consolidation of analytical methods, the interpretation of statistical concepts, debugging, and the preparation of technical documentation.

The collaboration combined several distinct layers:

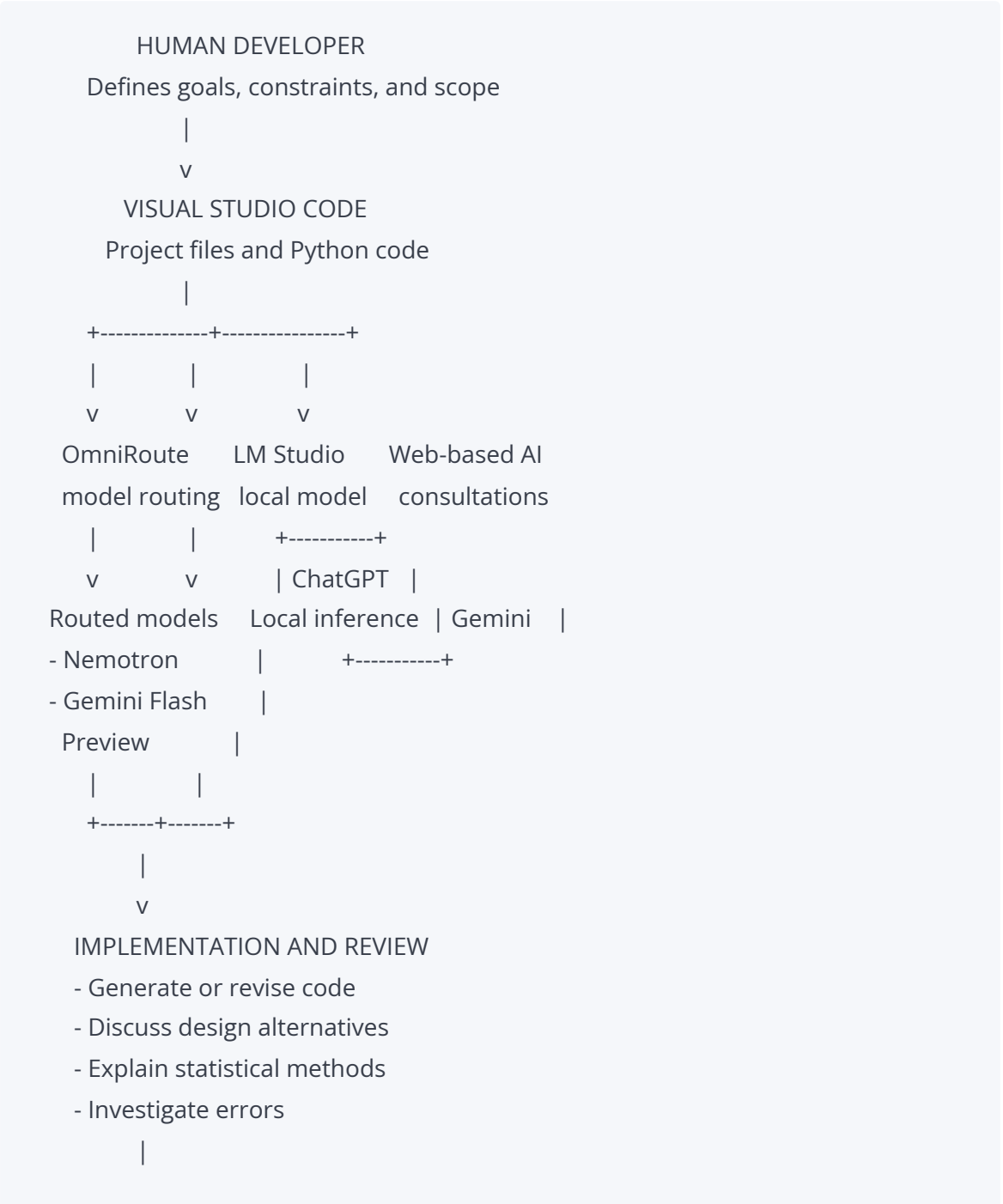
1. **Development environment:** Visual Studio Code for working with project files and Python scripts.
2. **Conversational AI:** The ChatGPT desktop application for iterative problem-solving, code generation and review, statistical explanations, and documentation.
3. **Additional AI-assisted development tools:** The reported workflow also included VS Code-based model routing through OmniRoute, Nemotron, and Gemini Flash Preview, alongside a local model served through LM Studio and web-based ChatGPT and Gemini consultations. Their specific contributions should be attributed to individual tasks only when supported by the corresponding interaction history.
4. **Data generation and analysis:** Python scripts for synthetic log generation, data preprocessing, statistical calculations, and visualization.
5. **Human oversight:** The developer retained responsibility for scope, analytical choices, interpretation, prioritization, and final conclusions.

The key principle was that AI accelerated implementation and exploration, while the developer determined what questions to investigate and how the evidence should be interpreted.

3. Architecture of the tools and analytical workflow

Two complementary diagrams describe the solution: the AI-assisted engineering environment and the runtime architecture of the telemetry prototype.

3.1 AI collaboration and development architecture



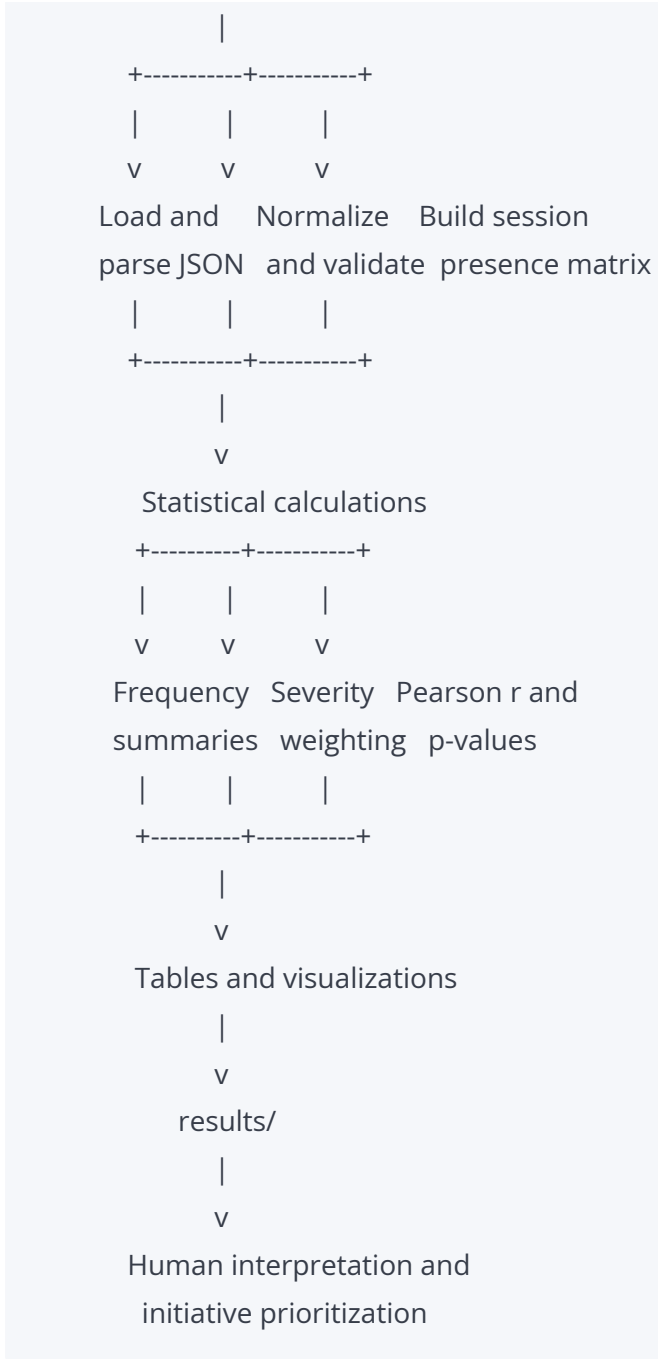
```
          v
  HUMAN REVIEW AND TESTING
          |
          v
  PYTHON ANALYTICAL PIPELINE
          |
          v
RESULTS, TABLES, AND FIGURES
          |
          v
  HUMAN INTERPRETATION
          |
          v
  FINAL TECHNICAL REPORT
```

Architecture note: This diagram represents the reported collaboration environment, not a claim that every model communicated directly with every other model. OmniRoute is represented as a routing layer, LM Studio as a local inference service, and web-based AI applications as separate consultation channels. The exact model assignments, routing rules, and execution paths depend on the configuration used during each task.

The diagram also does not imply that AI models independently validated the final results. Statistical outputs still required examination through code, data, and appropriate methodological checks.

3.2 Runtime architecture of the telemetry prototype

```
    CONFIGURATION AND SCENARIO
  Session count, duration, probabilities
                |
                v
        generate_logs.py
  Synthetic event and session generation
                |
                v
        system_signals.log
          JSON Lines
                |
                v
         analyze.py
```



The analytical pipeline is separated into data generation, preprocessing, statistical analysis, and reporting. This separation makes it possible to change simulation parameters, rerun the analysis, and examine whether the observed patterns remain consistent under alternative scenarios.

3.3 Separation of responsibilities

Component	Responsibility
Developer	Defines the problem, scope, constraints, acceptance criteria, and final interpretation.

Visual Studio Code	Provides the working environment for editing and organizing project files.
OmniRoute and routed models	Provide model access and AI-assisted responses through the reported routing configuration.
LM Studio	Provides a local inference endpoint for the configured local model.
ChatGPT desktop application	Supports iterative discussion, code assistance, statistical explanations, and documentation.
Web-based ChatGPT and Gemini	Provide additional consultation channels for questions, alternatives, and explanations.
Python	Executes the generator, preprocessing, statistical calculations, and visualization code.
pandas and SciPy	Support data manipulation and statistical calculations, including Pearson correlation.
Matplotlib and Seaborn	Support the creation of analytical figures and heatmaps.
Human review	Checks whether code, assumptions, outputs, and conclusions are appropriate for the task.

These responsibilities describe the tools' roles in the workflow. They do not establish that every listed tool was used in every stage or that every model produced code that was ultimately retained.

4. How each AI and development tool was used

4.1 Visual Studio Code

Visual Studio Code served as the development environment for working with the Python scripts and project artifacts.

Its role was to provide a place to organize, inspect, modify, and run the code. AI-generated suggestions could then be incorporated into the implementation and reviewed in the context of the existing files.

The editor should not be confused with the AI model itself: it is the working environment in which the code and configuration are maintained.

4.2 OmniRoute and routed models

The reported VS Code environment included OmniRoute and access to Nemotron and Gemini Flash Preview.

The routing layer was part of the AI-assisted development setup, allowing the developer to work with different model options instead of relying exclusively on a single conversational interface.

Potential uses of this arrangement include comparing proposed implementations, requesting alternative explanations, and selecting an appropriate model for a particular task. However, individual outputs should be attributed to a specific model only when the routing logs or interaction history establish that attribution.

No claim is made that the models were automatically benchmarked against each other or that routing decisions were always determined by cost or performance.

4.3 LM Studio and local model inference

LM Studio was included in the development environment to serve a locally installed language model.

```
VS Code or local client
        |
        v
Configured model client
        |
        v
LM Studio local endpoint
http://127.0.0.1:1234/v1
        |
        v
Locally loaded language model
        |
        v
Generated response
```

The endpoint above is the documented local endpoint for the development setup. The exact model identifier and its use in each task should be confirmed against the local configuration.

A local inference option can reduce dependence on paid hosted inference and allow experimentation without sending every prompt to an external model provider. It is not automatically free in total cost: hardware, electricity, latency, and maintenance still matter.

Local execution also does not guarantee that every interaction remains local. That depends on which client, endpoint, and model are used for each request.

4.4 ChatGPT desktop application

ChatGPT was the primary documented conversational interface for this journal.

It supported iterative work rather than a single prompt-and-response interaction. The developer could introduce a requirement, inspect a proposal, identify an omission, request a correction, and refine the implementation or explanation.

The main activities included:

- Discussing the engineering scenario and its constraints.
- Exploring how telemetry could be represented as structured data.
- Requesting Python implementation ideas and revisions.
- Understanding Pearson correlation, p-values, and heatmaps.
- Examining how statistical findings should be qualified.
- Improving the organization and readability of the technical report.
- Consolidating the AI collaboration record.

ChatGPT's responses were treated as assistance, not as an independent source of production evidence or proof of correctness.

4.5 Web-based ChatGPT and Gemini

Web-based ChatGPT and Gemini were additional consultation channels in the reported workflow.

They provided access to AI assistance outside the local development environment. Such channels can be useful for exploring alternatives, clarifying technical concepts, and comparing explanations.

The journal distinguishes the existence of these channels from task-level attribution. Unless a specific conversation or output is available, it does not claim that a particular model authored a particular function, detected a specific defect, or validated a statistical conclusion.

4.6 Python and analytical libraries

Python was the execution layer for the prototype. Unlike the conversational models, Python ran the deterministic portions of the pipeline against the generated data.

Tool or library	Role in the prototype
Python	Executes log generation, data processing, calculations, and output generation.
JSON Lines	Stores one structured JSON event per line in `system_signals.log`.
pandas	Loads, normalizes, groups, and transforms event records.
SciPy	Calculates Pearson correlation coefficients and associated p-values.
Matplotlib	Provides plotting functionality.
Seaborn	Produces statistical heatmaps and related visualizations.

The use of these libraries made the workflow executable and repeatable. The validity of the conclusions nevertheless depends on the data-generating process, implementation, assumptions, and interpretation.

5. Human–AI interaction record

Stage	Human contribution	AI-assisted contribution	Outcome and human control
Understanding the challenge	Identified the constraints of zero additional budget, no extra headcount, and a maximum of one or two initiatives.	Helped translate the scenario into an analytical prototype.	Synthetic telemetry was accepted as a way to test the analytical workflow, not as production evidence.
Data generation	Selected the five signal categories and requested configurable scenarios.	Assisted with the structure of a JSON Lines generator containing sessions, timestamps,	`generate_logs.py` and `system_signals.log` were created as prototype artifacts.

		severities, and event details.	
Severity analysis	Requested counts, percentages, and severity-weighted scores.	Assisted with aggregation and ranking logic.	The weights were treated as a heuristic rather than a measured business cost.
Correlation analysis	Asked whether relationships between signal categories could inform prioritization.	Explained session-level presence matrices, Pearson correlation, heatmaps, and p-values.	Correlation was included as an exploratory analysis, without claiming causality.
Statistical interpretation	Requested explanations of the numerical outputs and their implications.	Explained how coefficients and p-values should be read.	Interpretations were limited to the simulated dataset and the implemented method.
Code organization	Requested consolidation and clearer separation of generation and analysis.	Assisted with implementation proposals and script organization.	The code was structured around a generator and an analytical entry point, with overlapping scripts still requiring cleanup.
Communication	Requested a clearer report for technical and executive audiences.	Assisted with structure, explanatory language, tables, and limitations.	The report distinguished observed outputs, assumptions, and recommendations.
AI journal	Requested an auditable record of the collaboration.	Helped organize the activities, responsibilities, and limitations into a consolidated document.	The developer retained responsibility for the final account and its accuracy.

6. Technical implementation

6.1 Synthetic telemetry generation

`generate_logs.py` centralizes scenario parameters in `CONFIG`, including session count, simulation duration, random seed, and event probabilities.

Each generated record can contain:

- `timestamp`
- `session_id`
- `signal_category`
- `severity`
- `details`

The JSON Lines format makes it possible to process each event independently while preserving the relationship between events belonging to the same session.

The generator also includes explicit conditional relationships between categories. For example, the reported implementation can generate `SYSTEM_LAG` after `TECH_DEBT`, and operational overhead after lag.

```
has_tech_debt = random.random() < tech_debt_config["probability"]

if has_tech_debt:
    # Record a TECH_DEBT event for the session.
    has_system_lag = (
        random.random() < lag_config["probability_from_tech_debt"]
    )

    if has_system_lag:
        # Record SYSTEM_LAG using the same session_id.
        has_ticket = (
            random.random() < overhead_config["probability_from_lag"]
        )
```

This design is useful for testing whether the analysis can recover patterns that were deliberately introduced into the simulation.

It also creates a critical interpretive limitation: observed associations may be consequences of the generator's own rules. The results cannot independently establish that the same relationships exist in an actual RMS deployment.

6.2 Data loading and normalization

The main analytical workflow in `analyze.py` includes `load_logs()` and `preprocess_dataframe()`.

The loading function reads JSON Lines records and handles invalid entries. Preprocessing validates required columns, normalizes text, and extracts `eval_ms` from `details` when available.

```
required_columns = ["session_id", "signal_category"]

df["session_id"] = df["session_id"].astype("string").str.strip()
df["signal_category"] = (
    df["signal_category"].astype("string").str.strip()
)
df["severity"] = (
    df["severity"].fillna("INFO").astype(str).str.strip().str.upper()
)
```

This is a reasonable starting point for a prototype because inconsistent formatting and missing severity values can otherwise affect aggregation.

For production use, the pipeline would benefit from a formal schema contract, explicit data-quality metrics, structured error reporting, and automated tests for malformed input.

6.3 Session-level correlation

The correlation analysis operates on sessions rather than treating every event as an independent observation.

The code builds a binary presence matrix:

```
session_matrix = (
    pd.crosstab(df["session_id"], df["signal_category"]) > 0
).astype(int)

r, p_value = pearsonr(x, y)
```

Each row represents a session, and each column represents a signal category. A value of `1` indicates that the category appeared at least once during that session.

The resulting matrix supports an exploratory question: which signal categories tend to appear in the same sessions?

Pearson correlation measures the strength and direction of linear association between the binary indicators. It does not establish a causal relationship, temporal order, or an underlying mechanism.

The analysis also generates correlation and p-value matrices, with heatmaps produced using Seaborn and Matplotlib.

P-values must be interpreted in the context of the simulation and its assumptions. They do not correct for the fact that the relationships may have been deliberately introduced by the generator. If multiple pairwise tests are interpreted inferentially, the analysis should also consider multiple-comparison adjustments.

6.4 Frequency, severity, and weighted scores

`analyze_signals.py` summarizes event frequency and cross-tabulates categories against severity.

The focused analysis uses the following weights:

```
severity_weights = {"ERROR": 10, "WARN": 3, "INFO": 1}

df["risk_weight"] = (
    df["severity"].map(severity_weights).fillna(1)
)

risk_scores = df.groupby("signal_category")["risk_weight"].sum()
```

The weighted score combines frequency and assigned severity into a ranking that can support a constrained prioritization discussion.

The score is a heuristic defined for the exercise. It is not a direct measurement of financial loss, engineering effort, customer impact, or causal risk.

A production application would require stakeholders to agree on the weights and validate whether the resulting ranking aligns with operational and business outcomes.

7. Analytical outputs and interpretation

The prototype can produce:

- Event counts and percentages by signal category.
- Severity distributions for `ERROR`, `WARN`, and `INFO`.
- Severity-weighted scores.
- Session-level Pearson correlation matrices.
- Corresponding p-value matrices.
- Frequency charts, severity summaries, and heatmaps.

These outputs provide a reproducible basis for formulating and comparing prioritization hypotheses.

For example, if `TECH_DEBT` and `SYSTEM_LAG` frequently co-occur in the synthetic dataset, the developer can investigate whether simplifying custom rules and improving search or bulk-action performance are promising initiatives.

That observation does not demonstrate that fixing technical debt will automatically eliminate lag or support tickets. Such a claim would require evidence from an actual system, an appropriately designed intervention, and measurements before and after the change.

7.1 Evidence and decision boundaries

Statement type	Appropriate interpretation
Directly computed output	A count, percentage, coefficient, or score produced by the implemented code for a particular run.
Simulation-dependent pattern	An association or distribution observed in the generated dataset.
Engineering hypothesis	A proposed explanation or intervention suggested by the observed pattern.
Business recommendation	A choice that also requires operational context, expected effort, risk, and business priorities.
Production conclusion	A claim that requires representative production evidence and suitable validation.

This distinction prevents an analytical prototype from presenting its own assumptions as independently verified facts.

8. Human decisions versus AI assistance

8.1 Decisions retained by the developer

The developer retained responsibility for:

- Choosing synthetic telemetry as the initial test environment.
- Selecting the five signal categories.
- Defining the limited-resource prioritization problem.
- Requesting frequency, severity, weighted scores, and correlation analysis.
- Deciding that three complementary mathematical analyses would add value to the investigation.
- Proposing the use of log files to simulate different processing scenarios and test the analytical prototype.
- Determining how the outputs should be interpreted and qualified.
- Reviewing the business context before selecting recommendations.
- Approving the final report and the limitations stated in it.

The decision to introduce the three mathematical analyses was a human engineering decision. AI assisted with implementation and explanation, but the developer determined that these methods were relevant to the analytical objective.

Likewise, the proposal to create configurable logs was intended to test the processing pipeline under controlled scenarios, not to manufacture evidence of production behavior.

8.2 Activities supported by AI

AI-assisted work included:

- Proposing Python functions and data structures.
- Helping organize log generation, JSON Lines loading, and preprocessing.
- Consolidating related analytical methods into a more coherent workflow.
- Explaining Pearson correlation, p-values, and heatmaps.
- Suggesting visualizations and table formats.
- Helping investigate implementation issues and inconsistencies.
- Improving the report's structure and explanatory language.
- Consolidating the interaction history into an AI journal.

AI assistance did not transfer accountability for code correctness, statistical assumptions, or final recommendations to the model.

9. Limitations and quality controls

1. **Synthetic data:** The generator's rules influence observed frequencies and correlations.
2. **Correlation is not causation:** Pearson measures linear association, not cause, mechanism, or direction of influence.
3. **Binary session matrix:** The analysis measures co-occurrence within sessions rather than event sequence, duration, latency, or temporal causality.
4. **Heuristic weights:** Severity weights should be agreed with business and operations stakeholders before production use.
5. **Statistical inference:** P-values from a simulation do not establish real-world significance; multiple comparisons and the generator's dependencies must be considered.
6. **Code consolidation:** The project contains overlapping scripts, including `analyze.py`, `analyze_signals.py`, and `analyze_signal_correlations.py`. A production implementation should maintain a clearly defined entry point.
7. **Reproducibility:** Simulation configuration, random seed, code version, and execution date should be recorded for each run.
8. **AI attribution:** The presence of several models in the environment does not establish which model produced a particular implementation. Specific attribution requires supporting interaction records.
9. **Validation:** AI-generated code and explanations must be checked against actual execution, data, and the intended statistical method.
10. **Production readiness:** The prototype does not replace schema validation, automated testing, performance testing, monitoring, or engineering review.

10. Responsible next steps

1. Validate the hypotheses against representative production telemetry, if authorized and available.
2. Define success metrics for each candidate initiative, such as p95/p99 search latency, bulk-action failure rates, support-ticket volume, and adoption of feedback features.
3. Run a small, reversible intervention and compare appropriate pre- and post-intervention measurements.
4. Preserve the simulation configuration, code version, parameters, and outputs for each analytical run.
5. Consolidate the analytical scripts and introduce automated tests.
6. Document which AI tools contributed to significant implementation decisions when reliable records are available.
7. Maintain human review of AI-generated code, statistical interpretations, and business recommendations.

8. Avoid sending confidential or sensitive production data to external AI services unless the relevant authorization and data-handling requirements are satisfied.

11. Conclusion

AI-assisted development helped accelerate implementation, statistical exploration, debugging, and technical communication. The development process combined conversational assistance, a code editor, local and web-based model access, and executable Python analysis.

The developer retained ownership of the problem definition, analytical choices, simulation design, interpretation of results, and final prioritization.

The outcome is a prototype designed to test a reproducible telemetry-analysis workflow and formulate engineering hypotheses under constrained resources.

Its value lies in demonstrating a structured, human-directed approach to AI-assisted engineering—not in claiming that synthetic data establishes causal relationships or verifies production behavior.

The journal therefore treats AI as an engineering aid, statistical code as an executable method that requires validation, and human judgment as the final authority for interpreting evidence and making recommendations.

Recomendación antes de entregarlo: verifica que los nombres de los modelos y la configuración de OmniRoute coincidan con el entorno que utilizaste. Para que el journal sea realmente auditable, conserva ejemplos concretos de las interacciones que llevaron a decisiones importantes y, si los tienes, las versiones del código y los resultados de las pruebas. El diagrama muestra la arquitectura general sin afirmar que todos los modelos trabajaron entre sí automáticamente.